

000_Intro.ipynb

Oreum Reference - Copula Regression `oreum_copula`

Demonstrate Bayesian Copula Regression Modelling using Bayesian inference and a Bayesian workflow, specifically using the `pymc` & `arviz` ecosystem.

This **Intro** can also be used for verbal presentation and discussion purposes, ideally followed by a deeper technical walkthrough of the project in a long-form style. Because this project is a reference, it contains huge amounts of detail which is not worthwhile to summarise too much.

The interested reader should refer to the project notebooks where we evaluate the behaviour and performance of the models throughout the workflows, including several state-of-the-art methods unavailable to conventional max-likelihood / machine-learning models.

[PDF version](#)

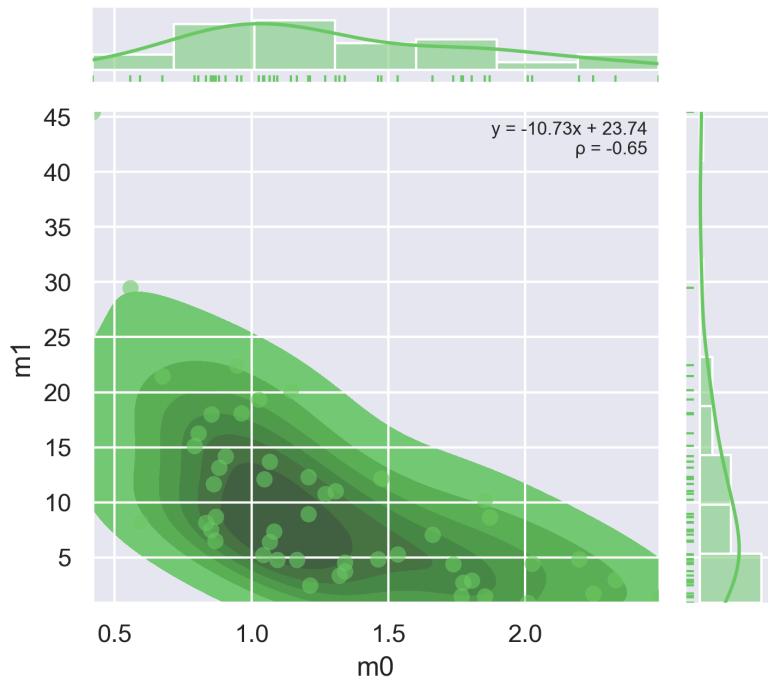
[Oreum Industries: Technical Resources](#)



What is Copula Regression?

We seek to create *principled* models that provide explanatory inference and predictions of Marginal distributions M that are jointly coupled by a Latent Copula C , using quantified uncertainty to support real-world decision-making.

Joint & marginal dists: `m0` vs `m1`, 50 obs
Observed Marginals with Copula Covariance



Motivation:

- A classic use-case for this model architecture (in the 2-dimensional setting) is insurance claims aka incurred loss
- We decompose the dollar value of claims into two marginal distributions: the **frequency**, and **severity** of **expected loss cost**, because these measures are intuitive and can behave differently, with a (highly important) degree of covariance Σ
- If we use a naive model that doesn't account for the covariance between **frequency** and **severity**, then the model predictions for **expected loss cost** can be hugely wrong!

Quick Aside on decomposition of claims **frequency** and **severity**

We can create different decompositions for different purposes, and according to the data available. A useful one shown here is the ratio of losses per unit TIV, to generalise to policies of different TIV

$$frq_i = \frac{claim_ct_i}{TIV_i},$$

$$sev_i = \frac{incurred_total_i}{claim_ct_i}$$

$$\mathbb{E}_{loss\ i} = frq_i * sev_i = \frac{incurred_total_i}{TIV_i}$$

where:

- Each policy $i \in n$ (the dataset of all policies) can have it's own (policy-level) frequency ($frq_i \geq 0$) and severity ($sev_i \geq 0$) of claim (and thus policy-level $\mathbb{E}_{loss\ i} \geq 0$)

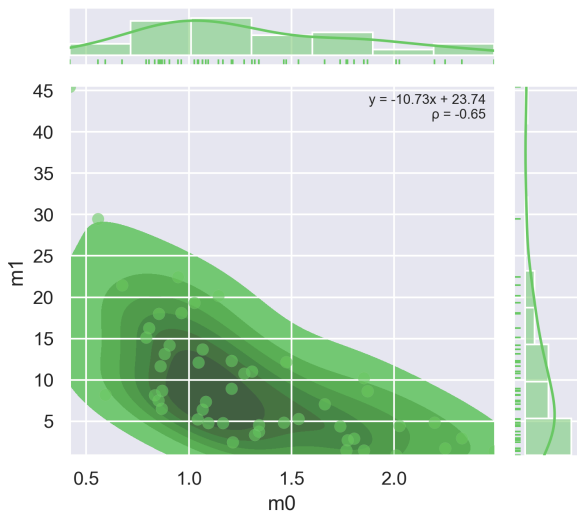
- Note frq and sev tend to be zero-augmented distributions (where no loss is experienced): this is a very important aspect to include in real-world model architectures
- $claim_ct_i \geq 0$ is the count of claims incurred for policy i
- $TIV_i > 0$ is the Total Insured Value (TIV) for policy i
- $incurred_total_i \geq 0$ is the total incurred losses for policy i

Plug:

- Oreum Industries is releasing a forecasting product **Vulcan** for DUA pricing & portfolio management that takes 3 simple tables (policy, premium_transactions, claims_transactions) and outputs probabilistic forecasts of Expected Loss Cost at policy-level
- At the heart of the solution is a holistic, principled, open-box Bayesian copula model (for covarying claims frq-sev) that runs quickly via API to integrate within modern data processing pipelines and dashboards
- We incorporate underwriter expertise; create hierarchical generalised linear regression on business-relevant parameters with high cardinality; allow for zero-claims inflation; auto-impute missing data; and more generally allow for small noisy datasets typical of DUA business.

Back to this presentation's focus on the copula function

Joint & marginal dists: `m0` vs `m1`, 50 obs
Observed Marginals with Copula Covariance



Demonstration:

- In this notebook:
 - We create a small synthetic dataset of observations of two marginals M_0, M_1 which have covariance Σ , and also (because we can) a version of the marginals M_{0x}, M_{1x} without covariance
 - We compare the resulting values of the joint product $y = M_0 * M_1$ vs $y = M_{0x} * M_{1x}$ and see that impact of ignoring the covariance is substantial.

- In the rest of the reference guide:
 - We create a series of principled copula models using advanced architectures and Bayesian inference to fit to the data and estimate the covariance on M_0, M_1
 - The first model is naive and ignores the covariance, the final model is very sophisticated and estimates the covariance
 - We demonstrate **a substantial 33 percentage-point improvement in model accuracy** when using a copula-based model
 - This correct estimation would likely make the difference between profitable pricing / accurate reserving, or greatly loss-making business over a portfolio.

General project approach

The emphasis in this project is to build a variety of models of increasing sophistication and demonstrate their usage. We strike a balance between building up concepts & methods vs practical application & worked examples in a `pymc` -based Bayesian workflow.

We don't focus on specific analysis of the dataset, nor try to infer too much. The dataset is simply a good substrate on which to learn and demonstrate the variety of model architectures used herein.

We evaluate the behaviour and performance of the models throughout the workflows, including several state-of-the-art methods unavailable to conventional max-likelihood / machine-learning models

This series of Notebooks covers

- `000_Intro.ipynb` : Orientation and fundamental concepts
- `100_ModelA0.ipynb` : Core (naive) architecture: Create priors, marginal likelihoods, but no copula
- `101_ModelA1.ipynb` : Partial architecture (extends ModelA0): Include Gaussian copula (w/ Jacobian adjustment), and several technical innovations to let `pymc` work with the transformations
- `102_ModelA2.ipynb` : Full architecture (extends ModelA1): Include Jacobian Adjustment on transformed observations

In this Notebook

We dive straight into **Orientation** and **Fundamental General Abstractions** with a simple real-world observational censored dataset, and then go on to demonstrate the theory and usage of an increasing sophistication of models.

Contents

- [Setup](#)
- [Preamble: Why Bayes?](#)

- 1. Orientation: Copula Functions and Their Behaviour
 - 2. Brief Technical Summary: The Copula Model designed in this Project
-

Setup

Imports

```
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from oreum_core import eda
from pyprojroot.here import here

# prepend local project src files
module_path = here('src').resolve(strict=True)
if str(module_path) not in sys.path:
    sys.path.insert(0, str(module_path))

from engine import logger
from synthetic.create_copula import CopulaBuilder

# autoreload local modules to allow local dev
%load_ext autoreload
%autoreload 2
import warnings # noqa
warnings.simplefilter(action='ignore', category=FutureWarning) # noqa
warnings.simplefilter(action='ignore', category=UserWarning) # noqa
import seaborn as sns
```

Notebook config

```
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

log = logger.get_logger('000_Intro', notebook=True)
_ = logger.get_logger('oreum_core', notebook=True)
```

Local Functions and Global Vars

```
RSD = 42
RNG = np.random.default_rng(seed=RSD)
```

Data Connections

```
figio = eda.FigureIO(here(Path('plots')).resolve(strict=True))
```

Preamble: Why Bayes?

We gain massive advantage by using a Bayesian Framework

We specifically use **Bayesian Inference** rather than Frequentist Max-Likelihood methods for many reasons, including:

	Bayesian Inference	Frequentist Max-Likelihood
	<i>Bayes' Rule</i>	
General formulation → Desirable Trait ↓	$P(\hat{\mathcal{H}} D) = \frac{\overbrace{P(D \mathcal{H})}^{\text{likelihood}} \cdot \overbrace{P(\mathcal{H})}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$	<i>MLE</i> $\hat{\mathcal{H}}^{\text{MLE}} \propto \arg \max_{\mathcal{H}} P(D \mathcal{H})$
Principled model structure represents hypothesis about the data-generating process	Very strong Can build bespoke arbitrary and hierarchical structures of parameters to map to the real-world data-generating process.	Weak Can only state structure under strict limited assumptions of model statistical validity.
Model parameters and their initial values represent domain expert knowledge	Very strong Marginal prior distributions represent real-world probability of parameter values before any data is seen.	Very weak No concept of priors. Lack of joint probability distribution can lead to discontinuities in parameter values.
Robust parameter fitting process	Strong Estimate full joint posterior probability mass distribution for parameters - more stable and representative of the expectation for the parameter values. Sampling can be a computationally expensive process.	Weak Estimate single-point max-a-posteriori-likelihood (density) of parameters - this can be far outside the probability mass and so is prone to overfitting and only correct in the limit of infinite data. But optimization method can be computationally cheap.
Fitted parameters have meaningful summary	Very strong Full marginal probability distributions can be interpreted exactly as probabilities.	Weak Point estimates only have meaningful summary statistics under strict limited assumptions of model statistical validity.

	Bayesian Inference	Frequentist Max-Likelihood
statistics for inference		
continues ...		
... continued		

Desirable Trait	Bayesian Inference	Frequentist Max-Likelihood
Robust model evaluation process	Strong Use entire dataset, evaluate via Leave-One-Out Cross Validation (best theoretically possible).	Weak Cross-validation rarely seen in practice, even if used, rarely better than 5-fold CV. Simplistic method can be computationally cheap.
Predictions made with quantified variance	Very strong Predictions made using full posterior probability distributions, so predictions have full empirical probability distributions.	Weak Predictions using point estimates can be bootstrapped, but predictions only have interpretation under strict limited assumptions of model validity.
Handle imbalanced, high cardinality & hierarchical factor features	Very strong Can introduce partial-pooling to automatically balance factors through hierarchical priors.	Weak Difficult to introduce partial-pooling (aka mixed random effects) without affecting strict limited assumptions of model validity.
Handle skewed / multimodal / extreme value target variable	Very strong Represent the model likelihood as any arbitrary probability distribution, including mixture (compound) functions e.g. a zero-inflated Weibull.	Weak Represent model likelihood with a usually very limited set of distributions. Very difficult to create mixture compound functions.
Handle small datasets	Very strong Bayesian concept assumes that there is a probable range of values for each parameter, and that we evidence our prior on any amount of data (even very small counts).	Very weak Frequentist concept assumes that there is a single true value for each parameter and that we only discover that value in the limit (of infinite observations).
Automatically impute missing data	Very strong Establish a prior for each datapoint, evidence on the available data within the context of the model, to automatically impute missing values.	Very weak No inherent method. Usually impute as a pre-processing step with weak non-modelled methods.

Practical Implementations of Bayesian Inference

We briefly referenced *Bayes Rule* above, which is a useful mnemonic when discussing Bayesian Inference, but in practice the crux of putting these advanced statistical techniques into practice is estimating the evidence $P(D)$ i.e. the probability of observing the data that we use to evidence the model

$$\underbrace{P(\hat{\mathcal{H}}|D)}_{\text{posterior}} = \frac{\overbrace{P(D|\mathcal{H})}^{\text{likelihood}} \cdot \overbrace{P(\mathcal{H})}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

...where:

$$P(D) \sim \int_{\Theta} P(D, \theta) d\theta$$

This joint probability $P(D, \theta)$ of data D and parameters θ requires an almost impossible-to-solve integral over parameter-space Θ . Rather than attempt to calculate that integral, we do something that sounds far more difficult, but given modern computing capabilities is actually practical.

We use a Bleeding-edge MCMC Toolkit for Bayesian Inference: `pymc` & `arviz`

We use **Markov Chain Monte-Carlo (MCMC)** sampling to take a series of *ergodic, partly-reversible, partly-randomised* samples of model parameters θ , and at each step compute the ratio of log-likelihoods $\log P(D|\mathcal{H})$ between a starting position (current values) θ_{p0} and proposed "sampled" position θ_p in parameter space, so as to reduce that log-likelihood (whilst exploring the parameter space).

This results in a posterior estimate $P(\hat{\theta}|D)$:

$$P(\hat{\theta}|D) \sim \frac{\overbrace{P(D|\theta_p)}^{\text{likelihood @ proposal}} \cdot \overbrace{P(\theta_p)}^{\text{prior @ proposal}}}{\underbrace{P(D|\theta_{p0})}_{\text{likelihood @ current}} \cdot \underbrace{P(\theta_{p0})}_{\text{prior @ current}}}$$

This is the heart of MCMC sampling: for detailed practical explanations see [Betancourt, 2021](#), [Carroll, 2019](#), and [Tweicki, 2015](#)

We use the bleeding-edge `pymc` and `arviz` Python packages to provide the full Bayesian toolkit that we require, including advanced sampling, probabilistic programming, statistical inferences, model evaluation and comparison, and more.



1. Orientation: Copula Functions and Their Behaviour

1.1 Create Synthetic Copula Dataset

We can learn a lot by creating a synthetic copula dataset using a "forward-pass":

1. Start with random data C sampled from the PDF of a Latent Copula function $\square_c$ with 2-dimensions

$$(C_0, C_1) \sim \square_c$$

2. Transform each dimension of the coupled data through the CDF of the copula function Φ_c to yield data distributed according to a Latent Uniform distribution U

$$(U_0, U_1) = \Phi_c(C_0, C_1)$$

3. Transform each dimension of now-uniform data through the Inverse CDF of our chosen marginal distribution Φ_m^{-1} to yield data distributed according to "observed" Marginal distribution(s) M

$$(M_0, M_1) = \Phi_m^{-1}(U_0, U_1)$$

In the following slides we'll plot the distributions and describe the transformations. Also see project class `synthetic.create_copula.CopulaBuilder` for details

Note we create 60 observations split into 2 sets: 50 for `train` (in-sample) and 10 for `holdout` (out-of-sample)

```
cb = CopulaBuilder()
df_all = cb.create(nobs=60)
cb.ref_vals
```

```
perm = RNG.permutation(df_all.index.values)
df_train = df_all.loc[perm[:50]]
df_holdout = df_all.loc[perm[50:]]
```

```
eda.describe(df_train, nobs=0, get_counts=False)
```

1.2 Visualise the Synthetic Observations

1.2.1 View the Latent Copula (an MvN)

In this forward-pass to create the synthetic data, we firstly create 50 observations of a 2-dimensional Multivariate Normal distribution with covariance Σ

$$(C_0, C_1) \sim \Phi_{\mathcal{C}} \\ \sim \text{MultivariateNormal}(\mu, \Sigma, \text{shape} = 2)$$

This forms our Latent Copula (a Gaussian), and this is where we could get creative and use any number of alternative copula functions from the literature (e.g. Clayton, Frank, Gumbel, etc) or even create our own: the copula marginals don't have to be the same distribution

```
f = eda.plot_joint_numeric(data=df_train, ft0='c0', ft1='c1', kind='kde+scatter', height=5,
    txtadd=f'Latent Copula = $MvN(0, \Sigma=\{cb.ref_vals["c_cov"].flatten()})')
```

Observe:

- Note the standard `Normal(0, 1)` scaling on the marginals
- Note the empirically-observed correlation $\rho \approx -0.7$ as defined in `c_cov`

1.2.2 View the Uniform-Transformed Marginals

In this forward-pass to create the synthetic data, next we pass each dimension of the Latent Copula C through the CDF of it's own function $\Phi_{\mathcal{C}}$ to get a Latent Uniform distribution U

$$(U_0, U_1) = \Phi_{\mathcal{C}}(C_0, C_1) \\ = \text{NormalCDF}(C_0, C_1)$$

Regardless of the latent copula, this intermediate step will result in 2 Uniform marginals (which still have covariance)

```
f = eda.plot_joint_numeric(data=df_train, ft0='u0', ft1='u1', kind='kde+scatter', color='blue',
    height=5, txtadd='Latent Uniform Marginals with Copula Covariance')
```

Observe:

- Now the marginals are uniform, but the correlation remains

1.2.3 View the Observed Marginals `m0`, `m1` (post transformation)

In this forward-pass to create the synthetic data, next we pass each dimension of the Latent Uniform U through the Inverse CDF of the marginal distribution $\Phi_{\mathcal{M}}^{-1}$ to get the Marginal distribution(s) in M

$$(M_0, M_1) = \Phi_{\mathcal{M}}^{-1}(U_0, U_1) \\ = \text{LogNormalInvCDF}(U_0, U_1)$$

The marginal distribution(s) M can be anything. In practice we tend to use right-tailed distributions in the Exponential family, here a LogNormal. We can, of course, use different distributions on each marginal - there's no constraint to be the same - but we use the same ones here. This is the data that we would observe in the real-world dataset:

```
f = eda.plot_joint_numeric(data=df_train, ft0='m0', ft1='m1', kind='kde+scatter', color='red', height=5, txtadd='Observed Marginals with Copula Covariance')
fqn = figio.write(f, fn='000_jointplot_corr')
```

Observe

- Marginals now have unique long-tail distributions, and the correlation remains

1.2.4 View the Marginals M^x if they were synthesized without a Copula

In project class `synthetic.create_copula.CopulaBuilder` we also synthesize uncorrelated observations using the same transformation and final marginals M , so that we can visually compare the different effects.

$$\begin{aligned}(C_0^x, C_1^x) &\sim \text{Normal}(\mu, \sigma, \text{shape} = 2) \\ (U_0^x, U_1^x) &= \text{NormalCDF}(C_0^x, C_1^x) \\ (M_0^x, M_1^x) &= \text{LogNormalInvCDF}(U_0^x, U_1^x)\end{aligned}$$

Uncorrelated marginals M^x individually look the same as M . We have to look at the joint distribution to see the difference

```
f = eda.plot_joint_numeric(data=df_train, ft0='m0x', ft1='m1x', kind='kde+scatter', color='red', height=5, txtadd='Observed Marginals without Copula Covariance')
```

Observe

- Spherical joint distribution, no correlation between our marginals here

1.2.5 Overplot Marginals Correlated (M) vs Uncorrelated (M^x) to Highlight the Differences

```
dfp = pd.concat((df_train[['m0', 'm1']], df_train[['m0x', 'm1x']]\
                .rename(columns={'m0x': 'm0', 'm1x': 'm1'})),
                axis=0, ignore_index=True)
dfp['corr_kind'] = np.repeat(['y_corr', 'y_uncorr'], repeats=len(df_train))
f = eda.plot_joint_numeric(
    data=dfp, ft0='m0', ft1='m1', hue='corr_kind', kind='kde', kdefill=False, color='red', height=5, txtadd='Observed marginals with / without Copula Covariance')
fqn = figio.write(f, fn='000_jointplot_corr_vs_uncorr')
```

Observe

- The marginals look almost identical, but the joint distribution is very different
- We might say "so what?" because we can always jointplot our marginals M and see that there is correlation
- The huge impact is that these lead to a very different **joint product** $y...$

1.3 Compare the Impact on Joint Product y

If we build a model $\mathcal{H}$ that fits to marginals M , but does not account for the correlation, $\mathcal{H}$ will behave as if we fit it on uncorrelated marginals M^x . The predicted differences in M and M^x won't look too different on the marginals, but the joint products $y = M_0 \cdot M_1$ vs $y^x = M_0^x \cdot M_1^x$ can become very different:

```
dfp['joint'] = dfp[['m0', 'm1']].product(axis=1)
pal = sns.color_palette(['C2', 'C3'])
f = eda.plot_smrystat_grp(dfp, grp='corr_kind', val='joint', palette=pal, plot_outliers=True)
fqn = figio.write(f, fn='000_y_corr_vs_uncorr')
```

Observe:

- This customer diagnostic combination plot shows:
 - Pointplot (left): The bootstrapped sums $\sum_i y_i$ vs $\sum_i y_i^x$
 - Boxplot (center): The individual values y_i vs y_i^x
 - Countplot (right): The counts of observations i
- The (bootstrapped) sum of `y_uncorr` ($\mu \approx 700$) is almost always much higher than for `y_corr` ($\mu \approx 500$)
- This shows that if our model $\mathcal{H}$ were to estimate marginals correctly but ignore the covariance, it would erroneously mis-estimate the joint distribution total value y . Here that mistake is to overestimate.

View the overestimate $\delta = \sum_i y_i^x - \sum_i y_i$

Let's view the bootstapped overestimate `delta = y_uncorr - y_corr`

```
idx = dfp['corr_kind'] == 'y_corr'
dfpp = pd.DataFrame({'delta': dfp.loc[~idx, 'joint'].values - dfp.loc[idx, 'joint'].values})
g = eda.plot_smrystat(dfpp, val='delta', txtadd='joint product value', plot_outliers=True)
fqn = figio.write(f, fn='000_y_delta')
```

Observe:

If we imagine this to be a portfolio of 50 policies, and the value of interest is an Expected Loss Cost $y = \mathbb{E}_{\text{loss}}$, and the units are dollars, then:

- If we were to use a model that ignores covariance, we might get a portfolio estimate of $\mathbb{E}_{\text{loss}} \approx 200$ dollars **higher** than if we were to use a better model that handles covariance with a copula function
- This overestimate is a substantial $\frac{700}{500} \approx +40\%$ and would likely make the difference between profitable pricing / accurate reserving, or greatly loss-making business over the portfolio.

2. Brief Technical Summary: The Copula Model designed in this Project

Again, this **Intro** is for verbal presentation and discussion purposes only - ideally followed by a deeper technical walkthrough of the project in a long-form style. Because this project is a reference, it contains huge amounts of detail which is not worthwhile to summarise too much.

The interested reader should refer to the project notebooks where state the architecture in full, we evaluate the behaviour and performance of the models in a consistent Bayesian workflow, including several state-of-the-art methods unavailable to conventional max-likelihood / machine-learning models.

Here we can highlight a very tangible impact of our results of using a Copula model (`ModelA2`) vs a Naive model (`ModelA0`)

2.1 Brief Orientation on Model Workflow and Architecture

General Approach

- We create a synthetic dataset with 60 observations: these have exogenous values on 2 marginals M_0, M_1
- We create 3 models of increasing sophistication to estimate $\hat{M}_0, \hat{M}_1$ and thus the joint product $\hat{y} = \hat{M}_0 \cdot \hat{M}_1$
- The simplest naive model (`ModelA0`) does not include a copula function, and the most sophisticated model `ModelA2` does

...

- We define a training set of 50 random observations, fit the models, and view the forecasted predictions on a holdout set of 10 observations
- We fully evaluate the models in the project notebooks using a variety of sophisticated techniques including In-sample Prior & Posterior Retrodictive ECDF plots, LOO-PIT calculations & plots, and more conventional coverage, RSME and R2 calculations. This forecast on the holdout is *not* a formal model evaluation
- However for discussion and elucidation we can plot the [bootstrapped](#) sum of the actual values $\sum y_{\text{holdout}}$ and compare to the posterior predictions $\sum \hat{y}_{\text{holdout}}$ of the two models

General Architecture

In contrast to the "forward-pass" that we use to synthesize the data, for the model we must of course start with the only data that we have (the observed marginals M) and work in a "backwards-pass" toward the copula.

1. Define 2 marginal distributions ($\mathfrak{M}_0, \mathfrak{M}_1$) (here for convenience we use the same family (Lognormals) for each, so we will represent as simply $\mathfrak{M}$). Each marginal is parameterised by a linear submodel $\beta^T \mathbf{x}_{ij}$ to allow linear regression onto selected features j

$$\mathfrak{M} = \text{LogNormal}(\mu = \beta^T \mathbf{x}_{ij}, \sigma)$$

2. Transform each dimension of the observed marginal data M through the CDF of the marginal distribution(s) to yield data distributed according to a Latent Uniform distribution U

$$(U_0, U_1) = \Phi_{\mathfrak{M}}(M_0, M_1)$$

3. Transform each dimension of now-uniform data U through the Inverse CDF of the copula distribution(s) ($\mathfrak{C}_0, \mathfrak{C}_1$) (here for convenience we use the same family (Normal aka a Gaussian Copula) for each, so we will represent as simply $\mathfrak{C}$)

$$(C_0, C_1) = \Phi_{\mathfrak{C}}^{-1}(U_0, U_1)$$

4. Evidence the transformed data C against the copula function $\log \mathcal{L} \mathfrak{C}$
5. For stability and correctness, we also evidence at the marginals and minimise a Jacobian adjustment on the double-transformed data.
6. **Importantly**, and unlike other model specifications in the Bayesian literature, we preserve the full posterior distribution(s) all the way through the model specification, without ever having to collapse to point estimates

Math Specification

Marginals $M_k, k \in 0, 1$ both LogNormal, evidence likelihood:

$$\begin{aligned} \beta_{M_k}^{j0} &\sim \text{Normal}(\mu, \sigma) \\ \sigma_{M_k} &\sim \text{InverseGamma}(\alpha, \beta) \\ \mathcal{L} \hat{M}_k &\sim \text{LogNormal}(\beta_{M_k}^T \mathbf{x}_i^{jk}, \sigma_{M_k}) \end{aligned}$$

Transform observed M_k to Latent Uniform U_k via their assumed CDFs:

$$U_k = \text{m}_{\mathfrak{t}} \Phi(M_k)$$

Transform Uniform Latent Normal C_k via a Normal InverseCDF

$$C_k = \text{MvNormal}(\mu = 0, \sigma = 1) \Phi^{-1}(U_k)$$

Create a latent covariance structure Σ

$$L \sim \text{LKJCholesky}(2), R \sim \text{LKJCorr}(2)$$

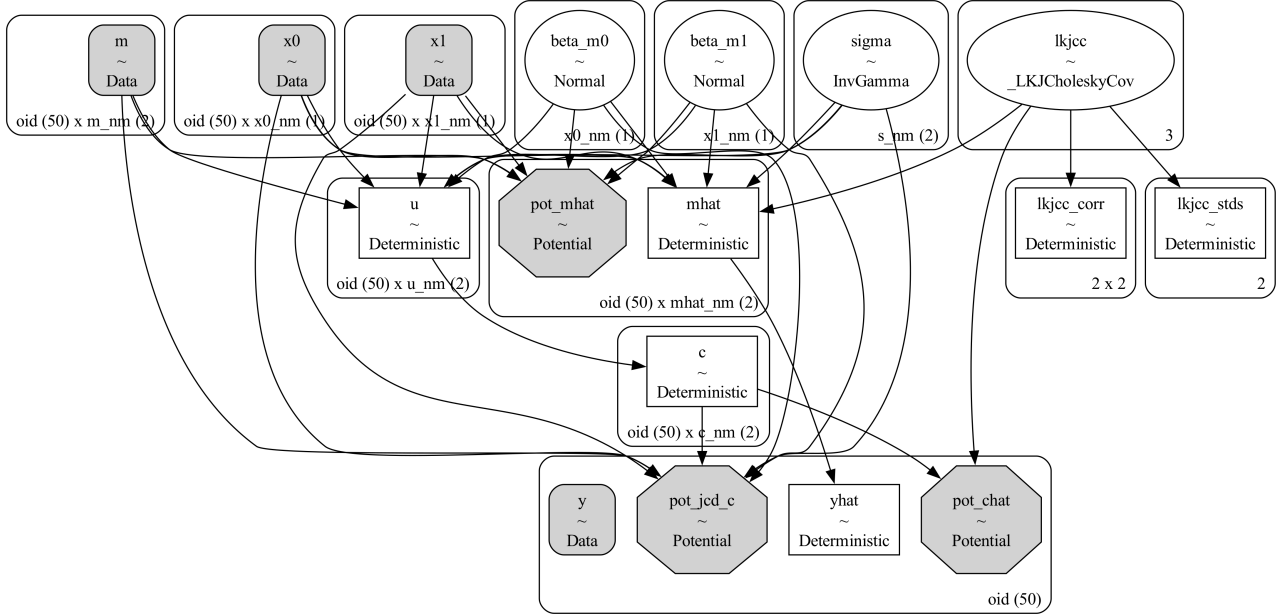
$$\sigma \sim \text{InverseGamma}(\alpha, \beta)$$

$$\Sigma \sim LL^T = \text{diag}(\sigma) * R * \text{diag}(\sigma)$$

Evaluate likelihood $\hat{C}$ of transformed marginals using MVNormal:

$$\mathcal{L} \hat{C} \sim \text{MvNormal}(\mu = 0, \Sigma, \text{observed} = C_k)$$

Plate Notation



Refer to Notebook `102_ModelA2.ipynb` and project class `models.copula.ModelA2` for full details.

This advanced, fully Bayesian architecture allows for:

- Regression via linear submodels $\beta^T \mathbf{x}$ on the marginals of `mhat`
- Efficient covariance via an `LKJCholeskyCovariance` structure
- A natural transformation from `M` $\rightarrow$ `U` $\rightarrow$ `C` including Jacobian Adjustment

2.2 Compare Estimated $\hat{y}$ ModelA0 vs ModelA2

ModelA0

This model sets a baseline for performance: it uses the same marginals but does not have a copula. This is "the best that one could do" with a naive non-copula architecture, and the performance / results are analogous to M^x that we discussed in §1

```
figio.read(fn='100_2_8_4_ppc_holdout_y_boxplot_mdla0_v1_3_0_dfx_holdout.png', figsize=
```

Observe:

- Now we can clearly see the impact: although the in-sample model fit was acceptable, the combined value y is way off, because this model ignores copula correlation between the marginals
- The mean of $\hat{y}$ is $\mu \approx 13.4$, is very different (and sits outside of) the bootstrapped actual data $\hat{y}$ is $\mu \approx 9.6$
- Comparing means we have a $\frac{13.4}{9.6} \approx +40\%$ overestimate!
- We do see that the PPC distribution envelops the bootstrapped actual data, which is promising, and means the model wouldn't necessarily be wrong to use, but there is clearly room to improve!

ModelA2

This is our most advanced model with a copula architecture: the performance / results are analogous to M that we discussed in §1

```
figio.read(fn='102_2_8_4_ppc_holdout_y_boxplot_mdla2_v1_2_0_dfx_holdout.png', figsize=
```

Observe:

- Now we can clearly see the impact: the Jacobian adjustment has allowed `mdla2` to estimate a much more precise and accurate value for $\hat{y}$
- The mean of $\hat{y}$ is $\mu \approx 10.4$, and maps nicely with the bootstrapped actual data $\hat{y}$ is $\mu \approx 9.6$
- Comparing means, we get $\frac{10.4}{9.6} \approx +7\%$ overestimate
- This is substantially better than `mdla0`, and also meaningfully improves on `mdla1`

ModelA2 vs ModelA0

In the above, we see a reduction in the mean overestimate of y from 40% down to 7%: a **33 percentage point drop** and 83% relative improvement!

This is a **huge difference** on this very small and simple dataset, and found only by correctly modelling the covariance using a copula and a sophisticated model architecture.

2.3 Supercharged Predictions with Quantified Uncertainty: Exceedance Curve

Now we pull back to demonstrate the power and utility of using a Bayesian model in the first place: because the predicted output values individually & jointly have **quantified uncertainty** aka empirical

probability. We can sum these and create an **Exceedance Curve (1 - ECDF)**

```
figio.read(fn='102_2_8_4_ppc_holdout_y_exceedance_mdla2_v1_2_0_dfx_holdout.png', figs:
```

We can read this curve *horizontally* to determine the value y at a particular probability P :

- $P_{@0.50} \hat{y} \geq 10.0$, (aka $\hat{y} \approx 10.0$ @ 1-in-2), and much closer to the actual data
- $P_{@0.05} \hat{y} \geq 14.8$, (aka $\hat{y} \approx 14.8$ @ 1-in-20)

Or we can read this curve *vertically* to determine the empirical probability P of achieving a particular value y :

- $P(\hat{y} > 15) \approx 0.05$: i.e. if we're worried about $\hat{y} > 15$, we might be less concerned because the probability is approx 1-in-20
- $P(\hat{y} > 17) \approx 0.01$: i.e. if we're worried about $\hat{y} > 17$, we might be sanguine because the probability is approx 1-in-100

This appears substantially tighter and more robust than `mdla0` and `mdla1`

Notes

```
%load_ext watermark
%watermark -a "jonathan.sedar@oreum.io" -udtmv -iv
```

Next Steps

Now the interested reader should dig into the Notebooks in project reference `oreum_copula`

There we demonstrate the full E2E workflow for models of increasing sophistication, including several state-of-the-art methods unavailable to conventional max-likelihood / machine-learning models.

- `100_ModelA0.ipynb` : Core (naive) architecture: Create priors, marginal likelihoods, but no copula
- `101_ModelA1.ipynb` : Partial architecture (extends ModelA0): Include Gaussian copula (w/ Jacobian adjustment), and several technical innovations to let `pymc` work with the transformations
- `102_ModelA2.ipynb` : Full architecture (extends ModelA1): Include Jacobian Adjustment on transformed observations

